

Towards Intent-based Configuration for Network Function Virtualization using In-context Learning in Large Language Models

Nguyen Van Tu
Dept. of CSE
POSTECH
Pohang, Korea
tunguyen@postech.ac.kr

Jae-Hyoung Yoo
Dept. of CSE
POSTECH
Pohang, Korea
jhyoo78@postech.ac.kr

James Won-Ki Hong
Dept. of CSE
POSTECH
Pohang, Korea
jwkhong@postech.ac.kr

Abstract— Network Function Virtualization (NFV) enables the execution of Virtual Network Functions (VNFs) on standard commodity servers. This brings flexibility, allowing for the rapid deployment of various network services while reducing costs. However, NFV configurations are becoming increasingly complex, necessitating experts for the setup. Intent-based network configuration has emerged as a solution to simplify NFV configuration and management. Nonetheless, it presents challenges, such as translating high-level natural language intents into low-level network configurations. In this work, we propose NFV-Intent - a system that leverages in-context learning in Large Language Models to perform the intent translation task. In-context learning enables NFV-Intent to work without retraining the Large Language Models, which is a difficult and expensive task. NFV-Intent uses a JSON template as the desired output, allowing Large Language Models to learn with a small number of examples and enabling easy verification of the configuration. Our evaluation showed that the intent can be translated into JSON configuration with high accuracy. To demonstrate the feasibility of NFV-Intent, we implemented and integrated it into the NITestbed, our previously developed system for AI-based NFV lifecycle management.

Index Terms—Network Function Virtualization, Virtual Network Function, Intent-based Networking, In-context Learning, Large Language Model

I. INTRODUCTION

Nowadays, networking systems are becoming more complex to provide various services such as internet browsing, financing, messaging, video streaming, and conferencing, as well as the Internet of Things. Each service has different requirements for Quality of Service, such as bandwidth, delay, and security. Deploying and managing numerous networking components and middleboxes are necessary to provide these services. However, most telecommunication providers still rely on expensive and proprietary physical middleboxes, such as Firewall, Network Address Translation (NAT), Deep Packet Inspection (DPI), Intrusion Detection System (IDS), and Load Balancer (LB), from vendors. These middleboxes are costly to deploy, maintain, manage, and upgrade, especially with the rapid rollout of new services and their evolving requirements.

Network Function Virtualization (NFV) [1] has emerged as a solution to this problem. NFV decouples network functions from physical components, allowing them to run on standard commodity servers. The decoupled network functions, called Virtual Network Functions (VNFs), not only reduce the cost of proprietary middleboxes but also increase the flexibility of network configuration. NFV allows for chaining VNFs to form useful network services in Service Function Chains (SFCs) and enables faster development, deployment, and upgrading of VNFs. However, NFV also faces challenges, and one of them is the complexity of configuration, which demands expert-level administrators.

Intent-based network configuration [2], [3] can simplify NFV configuration. Users or network administrators can express their desired network configuration using high-level, possibly natural language, as an intent. The intent is then translated into low-level network configuration for execution. Intent-based configuration reduces the need for writing complex configuration files, commands, or programs, saving time and reducing potential errors. However, the intent translation step presents a significant challenge: designing a model that understands natural language intent and accurately translates it into network configuration instructions. Recent advances in Large Language Models (LLMs) [4] provide a promising solution to this challenge. LLMs can understand human language in complex sentences and paragraphs, generating low-level instructions accordingly.

In this manuscript, we present NFV-Intent, a prototype system allowing the configuration of VNFs and SFCs using natural language-expressed intents with Large Language Models. Applying LLMs to intent translation poses challenges. To apply an LLM to a specific purpose, it usually needs fine-tuning and re-training with a custom dataset. Dataset collection/creation and model re-training can be expensive and time-consuming. Even then, the outcome model is not guaranteed to meet performance expectations. Our contributions are as follows:

- We introduce NFV-Intent, a prototype system leveraging

in-context learning in LLMs for intent translation without the need for re-training and fine-tuning the LLMs. NFV-Intent uses a JSON-based template as the translation output, facilitating understanding and verification for both LLMs and users/administrators.

- We implemented and integrated NFV-Intent into the NI-testbed [5], a framework we developed for applying Artificial Intelligence (AI) to VNF and SFC life-cycle management.
- We evaluated several aspects of NFV-Intent with both OpenAI *chatGPT* (*gpt-3.5-turbo* model) and six other offline LLMs. The initial evaluation showed high accuracy with only several examples and acceptable processing time.

The rest of the manuscript is organized as follows. Firstly, we survey the related work about Intent-based NFV configuration as well as in-context learning in LLMs in Section II. Secondly, we present the design and implementation of NFV-Intent in Section III. Thirdly, we show the evaluation results in Section IV. Then, we discuss the limitations of NFV-Intent and future work in Section V. Finally, we conclude this manuscript in Section VI.

II. BACKGROUND AND RELATED WORK

A. Intent-based configuration for NFV

Several studies have explored the use of intent for NFV configuration. In [6], the authors demonstrated an intent-based system for NFV configuration in an ETSI NFV environment. The system utilized a high-level JSON-based template for intent expression, which was then mapped to the NFV Management and Orchestration (MANO) system for deployment.

In [7], the authors attempted to combine NFV and Intent-based networking, enabling the automation of VNF deployment in cloud-based environments. The system supported multi-tenancy and intent refinement, utilizing a high-level pseudo-code for intent expression.

In [8], the authors proposed an Over-the-top Intent-based framework for network slicing in 5G networks. The system allowed for the automation of network slicing using declarative intents, written in a specific format with a limited vocabulary. The intent was then mapped to predefined NFV blueprints, and the system used these blueprints for NFV deployment and configuration.

While these works employed intent for NFV configuration, they did not allow the use of natural language for intent expression. In [9], the authors proposed a solution for intent refinement that could translate intent into a set of configurations and establish a service function chain, allowing the use of a constrained set of natural language for intent expression.

Although some other Intent-based networking systems allow the use of full natural language intents [10]–[13], they usually require retraining the Natural Language Processing (NLP) model with a custom dataset. This process can be expensive

and time-consuming, and even then, the performance outcome is not guaranteed. Our work explores another approach, in-context learning, which does not require retraining the model.

B. In-context learning in LLMs

Large Language Models (LLMs) have gained significant attention recently in both industries and academia with the introduction of *ChatGPT* [14]. Recent LLMs are trained with Internet-scale datasets and possess the ability to understand and interact with humans on various aspects and concepts.

In-context learning is a phenomenon in LLMs where the user provides the LLMs with several examples of input and output pairs. The LLMs then learn from these examples during the forwarding path (inference) of the model, without modifying the model weights. It differs from training, which updates the model weights.

Researchers have explored how in-context learning works [15], [16]. They found that LLMs learn many latent document-level concepts during training and then infer these during inference when there is a shared latent between examples and the actual input. They also provide guidelines on how to better leverage in-context learning to improve accuracy and reduce the number of examples needed.

III. DESIGN AND IMPLEMENTATION

A. Design Overview

NFV-Intent does not work as a standalone component; it is integrated into the NI-testbed, or Network Intelligence Testbed [5], an open-sourced platform for developing and experimenting with AI-based management methods for NFV. We developed NI-testbed on top of the OpenStack [17] platform. Figure 1 shows the architecture overview of NI-testbed. The detailed architecture and implementation are presented in [5].

NI-testbed contains NI-MANO for NFV management and orchestration functions, connecting to an OpenStack cloud infrastructure. NI-MANO consists of three main components: NI-NFVO, NI-MON, and NI-AI management modules. NI-MANO provides APIs for VNF and SFC deployment and management. NI-MON is in charge of monitoring VNFs, SFCs, and OpenStack infrastructure, providing information such as CPU, memory usage, and traffic load of all VNFs and OpenStack servers in real-time. NI-AI management consists of several AI-based modules, including auto-scaling, migration, and fault detection. Each AI module gets real-time information from NI-MON and automatically manages VNFs and SFCs when necessary by calling NI-NFVO APIs, forming a closed-loop control. A graphical interface is provided for initial VNF and SFC setup, as well as manual updating of VNF/SFC if needed. All the modules communicate via REST APIs.

Although NI-testbed has a closed-loop control while VNFs and SFCs are in operation, the initial configuration still needs an administrator to set up and configure using a GUI or command-line interface. Additionally, when VNFs or SFCs need to be manually changed, GUI or command-line interface

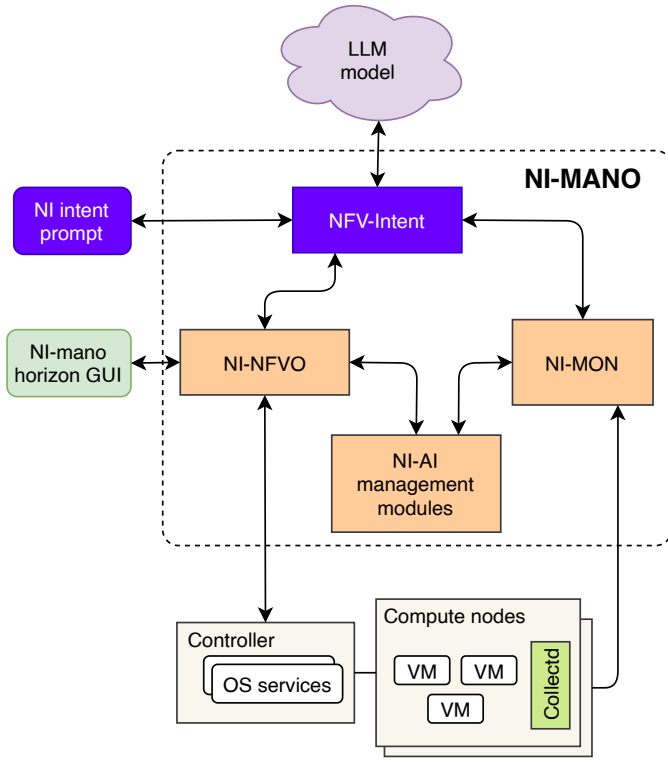


Fig. 1: NI-testbed architecture with NFV-Intent integrated

are the only two options. With the increasing complexity of the network, using a command-line interface is time-consuming and error-prone, while the GUI takes time to develop and also does not work well with complicated configurations.

NFV-Intent provides a simple natural language interface for configuring VNFs and SFCs in the testbed. Users express their intent through a prompt, such as 'create an SFC with firewall and IDS. The SFC should have a maximum bandwidth of 1Gbps. The SFC should route all traffic with port 80.' Subsequently, NFV-Intent sends this query to the LLM and retrieves the JSON configuration data for the intent. Finally, NFV-Intent interprets the JSON configuration and calls NI-NFVO to set up the VNFs and SFCs accordingly.

B. JSON template

We chose a JSON template as the output of intent translation because it has several advantages over configuration pseudo-code. Firstly, a JSON template is common and familiar to many administrators, making it easier to debug and work with compared to using new pseudo-code. Secondly, LLMs are trained on internet-scale data, which contains many JSON examples. Therefore, LLMs understand JSON templates easily compared to pseudo-code. Thirdly, JSON template configurations can be designed in a human-friendly and readable way, making the template easier to debug and verify. Fig. 2 shows a part of the NFV-Intent JSON template. The complete

JSON template should encompass all fields relevant to the configuration.

```
{
  "action": "create_vnf", # can be create_vnf,
    update_vnf, destroy_vnf, create_sfc,
    update_sfc, destroy_sfc
  "create_vnf": {
    "name": "vnf-name1",
    "vnf_type": "generic", # can be generic,
      fw, ids, lb
    "flavor": "1cpu.2gram.10gdisk",
    "num_instances": "auto"
  },
  "update_vnf": {
    "name": "vnf-name1",
    "vnf_type": "generic", #optional
    "flavor": "1cpu.2gram.10gdisk" #optional
  },
  "destroy_vnf": {
    "name": "vnf-name1"
  },
  "create_sfc": {
    "sfc_name": "sfc-name1",
    "max_bandwidth": "1Gbps",
    "delay_requirement": "normal", # can be
      normal, low-delay, real-time
    "flow_classifiers": [
      "0.0.0.0/0:0-65535 ->
        0.0.0.0/0:0-65535",
      "0.0.0.0/0:0-65535 ->
        0.0.0.0/0:0-65535"
    ],
    "vnfs": [
      {
        "vnf_type": "generic",
        "flavor": "1cpu.2gram.10gdisk",
        "num_instances": "auto"
      },
      {
        "vnf_type": "generic",
        "flavor": "1cpu.2gram.10gdisk",
        "num_instances": "auto"
      }
    ]
  }
}
```

Fig. 2: Part of NFV-Intent JSON template

In the current prototype, we support six actions, including creating, updating, and destroying VNFs; creating, updating, and destroying SFCs. When creating a VNF, a flavor is used to indicate the specification of the VNF, including VNF type, and the amount of CPU, memory, and disk that should be used. When creating SFCs, the QoS requirement, the traffic classifier, and the VNF chain should be defined. If the administrator does not specify the VNF specification in the intent during SFC expression, the flavor and the number of instances of each VNF in the chain are decided by the NI-NFVO module and AI auto-scaling module to meet the Quality of Service (QoS) requirement.

C. Prompt engineering

```

input: 'make a firewall with 2 CPU. The
       firewall name should be "example firewall"
       '
Output: {
  "action": "create_vnf",
  "create_vnf": {
    "name": "example firewall",
    "vnf_type": "fw",
    "flavor": "2cpu.2gram.10gdisk",
    "num_instances": "auto"
  }
}
input: 'create an SFC named "sfc1" with
       firewall and ids. It should have a maximum
       bandwidth of 500Mbps and no latency
       requirement. Apply traffic filtering on
       port 8080 and 22.'
Output: {
  "action": "create_sfc",
  "create_sfc": {
    "sfc_name": "sfc1",
    "max_bandwidth": "500Mbps",
    "delay_requirement": "normal",
    "flow_classifiers": [
      "0.0.0.0/0:0-65535 ->
        0.0.0.0/0:8080-8080",
      "0.0.0.0/0:0-65535 ->
        0.0.0.0/0:22-22"
    ],
    "vnfs": [
      {
        "vnf_type": "fw",
        "flavor": "1cpu.2gram.10gdisk",
        "num_instances": "auto"
      },
      {
        "vnf_type": "ids",
        "flavor": "1cpu.2gram.10gdisk",
        "num_instances": "auto"
      }
    ]
  }
}

```

Fig. 3: Examples of intent and JSON output in pre-prompt

To ensure that LLMs output the correct JSON configuration without unnecessary information, we use a pre-prompt. In this manuscript, we use the term pre-prompt to indicate the context that is appended to the message sent to LLMs before the actual intent, i.e., the JSON template and examples. The pre-prompt first instructs the model to only output JSON configuration and nothing else. Next, the pre-prompt contains the entire JSON template. We also add possible options for each field if it is not intuitive; for example, the possible options for *delay_requirement* are "normal," "low-delay," and "real-time". Finally, several examples of intents and corresponding JSON output are added to the pre-prompt. Figure 3 shows some examples that can be added to the pre-prompt.

D. Intent verification

The output JSON is not always deployable without conflict. For example, the intent may request a flavor that contains CPU or memory specifications not supported by the infrastructure. The intent may also violate the capability of the user; for instance, the intent may ask for a real-time low-delay connection, but the current user capability does not allow that. We include the verification in NFV-Intent and NI-MANO modules. If the intent is not deployable, the system should notify the administrator of the encountered error.

IV. EVALUATION

In this section, we evaluated two aspects: the accuracy of intent translation and the processing time from when the input intent is sent to when the complete output JSON configuration is received. We conducted the evaluation using a total of seven LLMs, as shown in Table. I. Out of the seven models, only *chatGPT* was run on the cloud using OpenAI's platform, while others were run offline using ollama [18], an open-source software to run LLMs locally. The offline models were executed on a machine equipped with two Intel Xeon Silver 4216 CPUs with hyper-threading enabled, 128GB of RAM, and an Nvidia Quadro RTX A6000 GPU with 48GB of VRAM.

TABLE I: List of LLMs used in evaluation

Model	Description
chatGPT [14]	OpenAI GPT-3.4 Turbo model
llama2 [19]	Open Foundation and Fine-Tuned Chat Models, 7B parameters version released by Meta
llama2:13b [19]	Open Foundation and Fine-Tuned Chat Models, 13B parameters version released by Meta
llava [20]	7B parameters Large Language and Vision Assistant model
mistral [21]	7B parameters model released by Mistral AI
openchat [22]	7B parameters model with Advancing Open-source Language Models with Mixed-Quality Data
neural-chat [23]	7B parameters model fine-tuned by Intel based on Mistral model

For verification, we needed a large test set that is not biased. We used a semi-automatic method to create the test set: we utilized Bing chat [24] to read the NFV configuration template and then asked Bing chat to generate 30 different intents and JSON configuration pairs based on the template. The intents were spread equally over six actions: "create_vnf," "create_sfc," "update_vnf," "update_sfc," "destroy_vnf," and "destroy_sfc." Then, we manually verified that the intent and JSON configuration were correct. However, the same intent can be expressed in many ways using natural language. To verify the accuracy of translating different expressions of the same intent, we used Bing chat to create 10 variations of each intent (i.e., change words, word orders, and the way of intent

TABLE II: Accuracy and processing time of seven LLMs on the test set when there are 9 examples in the pre-prompt

Model	chatGPT	llama2	llama2:13b	llava	mistral	openchat	neural-chat
Accuracy (%)	99.7%	92.0%	95.3%	77.6%	68.6%	86.0%	98.0%
Average processing time (ms)	4023	2619	2944	2554	3660	2548	2647
99 th processing time (ms)	11223	7348	8452	6923	13931	7314	7397

expression). In total, we have 300 pairs of intent and JSON configuration in the test set. We did not use Google Bard [25] for test set generation because, at the time of testing, Google Bard rephrased the intent incorrectly (i.e., the meaning of the intent was changed). Below are 4 examples of different variations of the same intent taken from the generated test set:

- Create an SFC named “sfc-1” with firewall and lb. It should have a maximum bandwidth of 1Gbps and low delay requirement. It should filter all incoming traffic on port 8080.
- Deploy an SFC called “sfc-1” including firewall and lb. Set the maximum bandwidth to 1Gbps with low delay requirement. Filter all incoming traffic on port 8080.
- Initiate the creation of an SFC named “sfc-1” with firewall and lb. Specify a maximum bandwidth of 1Gbps and low delay requirement. Apply filtering for all incoming traffic on port 8080.
- Generate an SFC named “sfc-1” containing firewall and lb. Configure a maximum bandwidth of 1Gbps with a low delay requirement. Apply filtering for all incoming traffic on port 8080.

There were several cases with extreme and awkward wording, such as “Initiate the creation of an SFC” in the third intent; however, we kept all of them in the test set without any change so that we could verify the ability to understand different ways of intent expressions.

A. Intent translation accuracy

For the evaluation, a pre-prompt containing the template and nine pairs of intent and JSON configuration examples was employed. The results in Table II showcase the accuracy and average processing time of each model. *ChatGPT* demonstrated the highest accuracy at 99.7%, with only one incorrect output out of 300 test cases. Among the offline models, *neural-chat* exhibited the highest accuracy at 98%, with six incorrect outputs out of 300 test cases. However, it’s crucial to note that the results do not necessarily imply superiority or inferiority of one model over another universally. Additionally, the test set generation process may introduce some bias toward *chatGPT*, especially since Bing chat, which employed the OpenAI gpt-4 model, was used in the test set creation.

Looking deeper into the test results, three types of failures were identified. The first type occurred when LLMs struggled with extreme cases of intent wording, involving rare words or awkward expressions. The second type of failure was observed

when one or several fields in the JSON configuration output were incorrect. The third type of failure happened when the output was not a valid JSON. For the best-performing model (*ChatGPT*), issues of the first and second types did not occur, and only one instance of the third type was identified. This underscores the importance of human verification in ensuring the correctness of the output JSON configuration.

B. Processing time

The delay from the time the query was sent by NFV-Intent to the time the JSON translation was received was measured. The experiment was conducted similarly to Section IV-A. The average processing time and 99th processing time of each model are presented in Table II. *ChatGPT*, running in the OpenAI cloud infrastructure, exhibited the highest average processing time, around 4 seconds. *Neural-chat*, the model with the highest accuracy among offline models, had an average processing time of approximately 2.6 seconds.

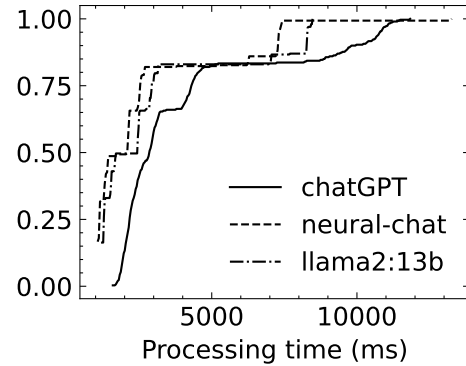


Fig. 4: Cumulative Distribution Function of processing time of three best-performed model: *chatGPT*, *neural-chat*, and *llama2:13b*, given nine examples in the pre-prompt.

Fig. 4 shows the Cumulative Distribution Function (CDF) of processing time of the three best-performing models: *chatGPT*, *neural-chat*, and *llama2:13b*. The 99th percentiles of processing time for *chatGPT*, *neural-chat*, and *llama2:13b* were 11.2 seconds, 7.3 seconds, and 8.4 seconds, respectively. We believe that both the average and 99th percentile of processing time are acceptable for the purpose of intent translation.

C. Effect of examples

In this test, we aimed to evaluate the impact of the number of examples on the accuracy of intent translation. Similar to the experiment in Section IV-A, we used a pre-prompt containing

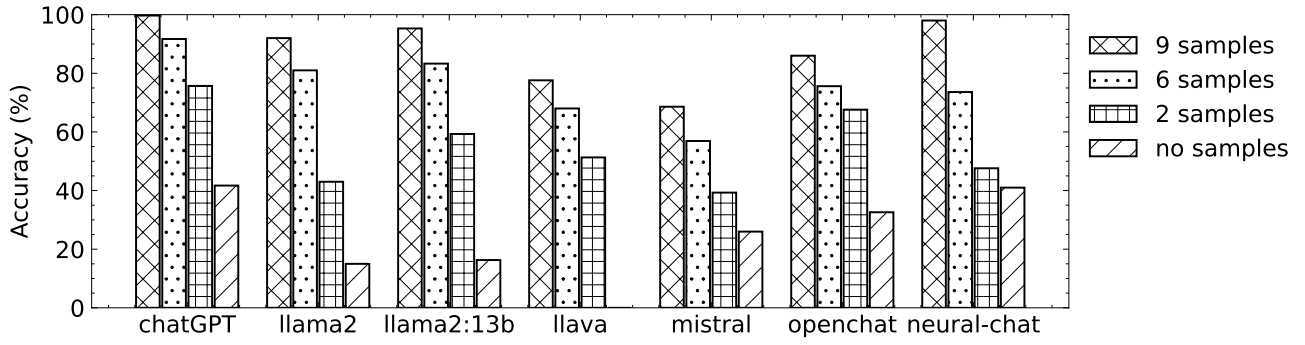


Fig. 5: Accuracy of *chatGPT* and 6 offline LLMs when changing the number of examples in the pre-prompt between 9, 6, 2, and no example.

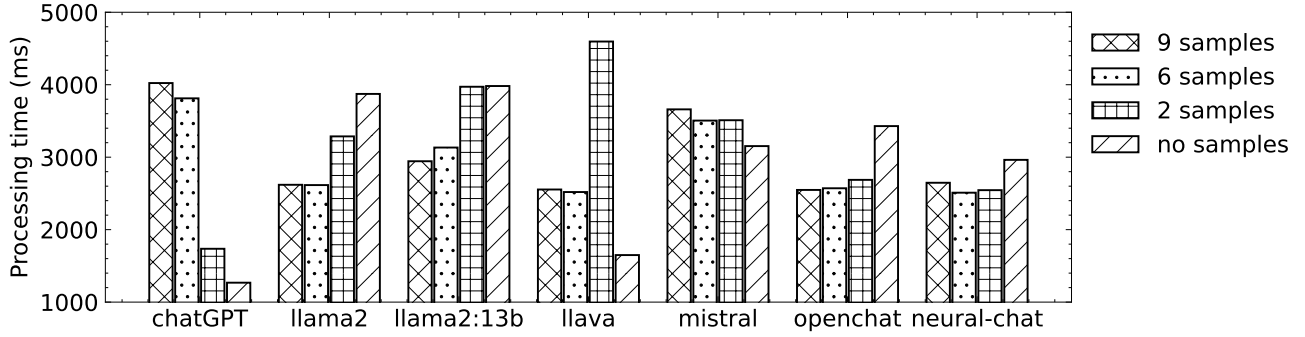


Fig. 6: Average processing time of *chatGPT* and 6 offline LLMs when changing the number of examples in the pre-prompt between 9, 6, 2, and no example.

6, 2, and no example intent and JSON configuration pairs. Fig. 5 illustrates the accuracy of seven LLMs in the test with varying numbers of examples in the pre-prompt. Although *chatGPT* maintained the highest accuracy throughout the changes, *neural-chat* exhibited a decrease in accuracy when compared to *llama2*, *llama2:13b*, and *openchat* as the number of examples reduced to six.

This test highlights that while LLMs can infer certain information correctly without examples, there is no assurance that they will do so consistently for every case. Hence, providing sufficient examples that cover all aspects of the template becomes crucial, particularly for fields that may not be intuitive.

Fig. 6 presents the average processing time of seven LLMs as the number of examples in the pre-prompt varies. For *chatGPT*, the processing time decreased when the number of examples was reduced, attributed to the reduction in the input context size (i.e., pre-prompt). Conversely, this trend was not observed for the remaining offline models. The average processing time of *llama2*, *llama2:13b*, *openchat*, and *neural-chat* increased when the number of examples decreased. Nevertheless, in all test cases, the average processing time remained below 5 seconds, which is acceptable for the purpose of intent translation.

V. LIMITATION AND FUTURE WORK

A. Limitation

Currently, the NFV-Intent's JSON template is simple and short, so LLMs can understand and work with it pretty easily. However, when the template becomes longer and more complex, the LLM may not be able to understand it and translate the intent correctly. More work and evaluation need to be done in that direction. Additionally, the current evaluation does not cover enough cases to ensure intent translations are always correct. We also need to do more evaluation on NFV-Intent with a larger number of intents and examples to verify its accuracy.

In the current state, NFV-Intent also has a limitation on the size of the pre-prompt. The maximum number of tokens in *gpt-3.5-turbo* is 4097 tokens, which may not be enough for a very long and complex template as well as a larger number of examples in the pre-prompt. For offline models, *llama2* has a context size of 4096 tokens, and *neural-chat* has a context size of 2048 tokens.

Currently, although NFV-Intent does catch several obvious issues in the generated JSON configuration (e.g., cannot be deployed to the network due to resource conflicts or not enough resources), the intent translation still needs to be verified by an expert before being deployed in the network.

For a fully automated network, a better verification system might need to be developed. However, building such a system is still a big challenge.

B. Future work

We have planned several directions to improve NFV-Intent. Firstly, we plan to design a complete JSON template and a complete example set that covers more functions. We also plan to design a more complete test suite to verify the functionality of the new template. This needs to be done along with solving the token limitation issue. To address the token limitation, the pre-prompts can be split into several messages. However, more experiments may need to be done to verify whether the model still works well in this case. Note that the issue can also be mitigated by using a newer model such as *gpt-4-32k* [14], which supports up to 32768 tokens in one message.

We also plan to investigate more on the intent verification. Currently, the verification is done in NFV-Intent, and NI-MANO only checks whether the intent has a hard conflict with the infrastructure, i.e., the deployment of the intent will cause a network error. Other types of soft conflicts, such as the intent being deployable but affecting the QoS of other intent, are not checked in the current state of NFV-Intent.

In this work, we do not consider the scalability of the system. However, in a large system, there may be many intents that need to be translated at the same time. We plan to evaluate the throughput of the LLMs in future work. Note that in the current test, we did not batch process the intents. The throughput of LLMs can be greatly improved if batch processing is used.

VI. CONCLUSION

Network Function Virtualization brings many benefits to modern network infrastructures, reducing the cost of deployment, operation, and management while making the network more flexible. However, along with the flexibility is the complexity of the configuration. To simplify the configuration of NFV, we propose NFV-Intent, a system that allows the use of natural language intents to do the NFV configurations. NFV-Intent uses Large Language Models to translate intents expressed in natural language into JSON configuration. By leveraging in-context learning in LLMs, the LLMs do not need to be fine-tuned or re-trained with custom datasets. We evaluated our intent translation approach with seven LLMs. The evaluation showed that using our approach, LLMs can translate intents with high accuracy using only a few examples. The processing time is also short enough for any type of intent. We implemented and integrated NFV-Intent into NI-testbed, a framework for building AI-based solutions for VNF life-cycle management, which showed the feasibility of our approach. While NFV-Intent is built for NFV configuration, we expect that our approach can be applied to other intent-based networking problems.

This prototype of NFV-Intent has several limitations and therefore we have several directions to improve it. We plan

to enhance NFV-Intent with a more complete template that supports more functionality. Additionally, we also need to investigate intent verification to detect wrong-translated intents or any type of soft conflict with the network and other already deployed intents. Finally, a scalability evaluation would be necessary before a large-scale deployment.

ACKNOWLEDGMENTS

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (2018-0-00749, Development of Virtual Network Management Technology based on Artificial Intelligence).

REFERENCES

- [1] R. Mijumbi, J. Serrat, J. L. Gorricho, N. Bouten, F. D. Turck, and R. Boutaba, "Network Function Virtualization: State-of-the-Art and Research Challenges," in *IEEE Communications Surveys Tutorials*, vol. 18, 2016, pp. 236–262.
- [2] A. Leivadreas and M. Falkner, "A survey on intent-based networking," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 625–655, 2023.
- [3] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, "Intent-Based Networking - Concepts and Definitions," RFC 9315, Oct. 2022. [Online]. Available: <https://www.rfc-editor.org/info/rfc9315>
- [4] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, "A survey of large language models," 2023.
- [5] S. Lange, N. V. Tu, S.-Y. Jeong, D.-Y. Lee, H.-G. Kim, J. Hong, J.-H. Yoo, and J. W.-K. Hong, "A network intelligence architecture for efficient vnf lifecycle management," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 1476–1490, 2021.
- [6] D. Borsatti, W. Cerroni, G. Davoli, and F. Callegati, "Intent-based service function chaining on etsi nfv platforms," in *2019 10th International Conference on Networks of the Future (NoF)*, 2019, pp. 144–146.
- [7] A. Leivadreas and M. Falkner, "Vnf placement problem: A multi-tenant intent-based networking approach," in *2021 24th Conference on Innovation in Clouds, Internet and Networks and Workshops (ICIN)*, 2021, pp. 143–150.
- [8] F. AKLAMAMU, S. RANDRIAMASY, and E. RENAULT, "Demo: Intent-based 5g iot application network slice deployment," in *2019 10th International Conference on Networks of the Future (NoF)*, 2019, pp. 141–143.
- [9] E. J. Scheid, C. C. Machado, M. F. Franco, R. L. dos Santos, R. P. Pfitscher, A. E. Schaeffer-Filho, and L. Z. Granville, "Inspire: Integrated nfv-based intent refinement environment," in *2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*, 2017, pp. 186–194.
- [10] H. Mahtout, M. Kiran, A. Mercian, and B. Mohammed, "Using machine learning for intent-based provisioning in high-speed science networks," in *Proceedings of the 3rd International Workshop on Systems and Network Telemetry and Analytics*, ser. SNTA '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 27–30. [Online]. Available: <https://doi.org/10.1145/3391812.3396269>
- [11] E. J. Scheid, P. Widmer, B. B. Rodrigues, M. F. Franco, and B. Stiller, "A controlled natural language to support intent-based blockchain selection," in *2020 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, 2020, pp. 1–9.
- [12] A. S. Jacobs, R. J. Pfitscher, R. H. Ribeiro, R. A. Ferreira, L. Z. Granville, W. Willinger, and S. G. Rao, "Hey, lumi! using natural language for Intent-Based network management," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, Jul. 2021, pp. 625–639. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/jacobs>
- [13] K. Dzevaroska, J. Lin, A. Tizghadam, and A. Leon-Garcia, "LLM-based policy generation for intent-based management of applications," in *2023 2023 19th International Conference on Network and Service Management (CNSM)*, 10 2023, pp. 1–7.

- [14] OpenAI, “OpenAI GPT models.” [Online]. Available: <https://platform.openai.com/docs/guides/gpt>
- [15] S. M. Xie, A. Raghunathan, P. Liang, and T. Ma, “An explanation of in-context learning as implicit bayesian inference,” 2022.
- [16] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, “Rethinking the role of demonstrations: What makes in-context learning work?” 2022.
- [17] Openstack, “Openstack: Open source cloud computing infrastructure,” 2023. [Online]. Available: <https://www.openstack.org/>
- [18] Ollama, “ollama: Get up and running with large language models locally.” 2023. [Online]. Available: <https://github.com/ollama/ollama>
- [19] Meta, “Llama 2: Open foundation and fine-tuned chat models,” 2023. [Online]. Available: <https://llama.meta.com/llama2/>
- [20] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *NeurIPS*, 2023.
- [21] M. AI, “Mistral,” 2023. [Online]. Available: <https://mistral.ai/news/announcing-mistral-7b/>
- [22] G. Wang, S. Cheng, X. Zhan, X. Li, S. Song, and Y. Liu, “OpenChat: Advancing Open-source Language Models with Mixed-Quality Data,” 2023.
- [23] Intel, “Neural-chat,” 2023. [Online]. Available: <https://huggingface.co/Intel/neural-chat-7b-v1-1>
- [24] Microsoft, “Bing chat,” 2023. [Online]. Available: <https://www.bing.com/chat>
- [25] Google, “Google bard,” 2023. [Online]. Available: <https://bard.google.com/chat>